Competitive Programming

Course Code: BCS24556B

Question Bank - Java Solutions, Line-by-Line Explanations and Dry Runs

Global Academy of Technology - Department of Computer Science and Engineering

Contents

Session 1: Data Types, Operators & Control Flow	Questions 1.1, 1.2
Session 2: Functions, Recursion & Memory Management	Questions 2.1, 2.2
Session 3: Pointers, Arrays & Strings	Questions 3.1, 3.2
Session 4: Structures, Unions & File Handling	Questions 4.1, 4.2
Session 5: Linked Lists in C	Questions 5.1, 5.2
Session 6: STL - Stacks, Queues & Priority Queues	Questions 6.1, 6.2
Session 7: Linked Lists, Binary Trees, Stacks & Queues	Questions 7.1, 7.2
Session 8: String Matching	Questions 8.1, 8.2
Session 9: Performance Analysis of Sorting and Searching	Questions 9.1, 9.2
Session 10: Number Theory	Questions 10.1, 10.2

Each question has: 1. Problem | 2. Java solution | 3. Line-by-line explanation | 4. Dry run

Every program was run against the sample test case from the question bank and gives the same output. Java has no pointers, malloc or unions, so questions written for C use the closest Java equivalent: ArrayList for dynamic memory, a class with an enum tag for the tagged union, and DataOutputStream / DataInputStream for the binary file.

Session 1: Data Types, Operators & Control Flow

Question 1.1

Single Unique ID Finder (Bitwise Operations)

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

In a smart warehouse, inventory tags are tracked using 32-bit integer IDs. Due to a system glitch, every tag ID in a log file appears exactly twice, except for one unique tag ID which appears only once. Write a C program to identify the unique tag ID.

Constraints / Requirements

- Your solution must run in O(N) time complexity and use O(1) auxiliary space.
- You are strictly required to use Bitwise XOR (^) operations without auxiliary arrays or hashing.

Input Format

First line contains an integer N (1 <= N <= 10^5), representing the number of elements in the array (always odd).
Second line contains N space-separated integers representing the tag IDs.

Output Format

Print the single integer that appears only once.

Sample Input	Sample Output
5 4 1 2 1 2	4

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          Scanner sc = new Scanner(System.in);
 6          int n = sc.nextInt();
 7          int result = 0;
 8          for (int i = 0; i < n; i++) {
 9              result ^= sc.nextInt();
10          }
11          System.out.println(result);
12      }
13  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import the java.util classes; we need Scanner for reading input.
3	`public class Main {`	Every Java program lives inside a class; this one is called Main.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner reads the input typed on the keyboard.
6	`int n = sc.nextInt();`	Read N, the number of IDs.
7	`int result = 0;`	Start from 0, because x ^ 0 = x.

Line	Code	Explanation
8	`for (int i = 0; i < n; i++) {`	Repeat N times, once for each ID.
9	`result ^= sc.nextInt();`	XOR the next ID into result. Equal IDs cancel out (a ^ a = 0), so only the unique ID is left.
10	`}`	End of the loop.
11	`System.out.println(result);`	Print the unique ID.
12	`}`	End of main.
13	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: N = 5 and IDs = 4 1 2 1 2. result starts at 0. In binary: 4 = 100, 1 = 001, 2 = 010.

i	data[i]	result before	result ^ data[i] (binary)	result after
1	4	0	000 ^ 100 = 100	4
2	1	4	100 ^ 001 = 101	5
3	2	5	101 ^ 010 = 111	7
4	1	7	111 ^ 001 = 110	6
5	2	6	110 ^ 010 = 100	4

Result

The pairs (1, 1) and (2, 2) cancelled out, leaving 4. Output: 4

Question 1.2

32-Bit Integer Reversal with Overflow Guard

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a signed 32-bit integer N, write a C function int reverseInteger(int n) that reverses its digits using basic arithmetic operations (% and /) and iterative loops.
If reversing N causes the value to overflow or underflow the 32-bit signed integer range [-2^31, 2^31 - 1], the function must return 0.

Input Format

A single signed 32-bit integer N.

Output Format

A single integer representing the reversed value, or 0 if an overflow occurs.

Sample Input	Sample Output
-1230	-321

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      static int reverseInteger(int n) {
 5          int rev = 0;
 6          while (n != 0) {
 7              int digit = n % 10;
 8              n = n / 10;
 9              long next = (long) rev * 10 + digit;
10              if (next > Integer.MAX_VALUE || next < Integer.MIN_VALUE) {
11                  return 0;
12              }
13              rev = (int) next;
14          }
15          return rev;
16      }
17
18      public static void main(String[] args) {
19          Scanner sc = new Scanner(System.in);
20          int n = sc.nextInt();
21          System.out.println(reverseInteger(n));
22      }
23  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`static int reverseInteger(int n) {`	Method that reverses the digits of n and returns 0 on overflow.
5	`int rev = 0;`	rev holds the reversed number as we build it.
6	`while (n != 0) {`	Repeat until no digits are left.
7	`int digit = n % 10;`	Last digit of n (it is negative when n is negative).
8	`n = n / 10;`	Remove the last digit from n.

Line	Code	Explanation
9	`long next = (long) rev * 10 + digit;`	Compute the next value in a 64-bit long so that it cannot overflow.
10	`if (next > Integer.MAX_VALUE || next < Integer.MIN_VALUE) {`	Is it outside the 32-bit signed range?
11	`return 0;`	Overflow or underflow, so return 0 as required.
12	`}`	End of the range check.
13	`rev = (int) next;`	It fits, so store it back in rev.
14	`}`	End of the loop.
15	`return rev;`	Return the reversed number.
16	`}`	End of reverseInteger.
18	`public static void main(String[] args) {`	The program starts running here.
19	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
20	`int n = sc.nextInt();`	Read the integer N.
21	`System.out.println(reverseInteger(n));`	Call the method and print the answer.
22	`}`	End of main.
23	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: n = -1230. In Java, % and / keep the sign, so the digits come out negative and rev is built directly as a negative number. rev starts at 0.

Loop	n (start)	digit = n % 10	next = rev*10 + digit	n after n / 10
1	-1230	0	0*10 + 0 = 0	-123
2	-123	-3	0*10 + (-3) = -3	-12
3	-12	-2	-3*10 + (-2) = -32	-1
4	-1	-1	-32*10 + (-1) = -321	0

Result

n = 0 ends the loop. rev = -321, and every value stayed inside the 32-bit range. Output: -321

Overflow example: n = 1534236469. At the last digit, next = 964632435 * 10 + 1 = 9646324351, which is more than 2147483647, so the method returns 0.

Session 2: Functions, Recursion & Memory Management

Question 2.1

Optimized Tail-Recursive Exponentiation with Callback Validation

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Implement a fast exponentiation function in C to compute x^n in O(log n) time using Tail Recursion.
Your solution must fulfill the following:
1. Define a function signature double power_tail(double x, long long n, double accumulator).
2. Define a callback function pointer typedef int (*validator_fn)(double, long long) that validates edge cases.
3. Write a wrapper function double my_pow(double x, long long n, validator_fn validate) that calls the tail-recursive function.

Input Format

A double value x representing the base and a signed 64-bit integer n representing the exponent separated by space.

Output Format

Print the result formatted to 5 decimal places. If an invalid math condition occurs (e.g., 0^-1), print INVALID.

Sample Input	Sample Output
2.00000 -10	0.00098

2 Java Solution

```
import java.util.*;

public class Main {
    interface Validator {
        boolean validate(double x, long n);
    }

    static double power_tail(double x, long n, double accumulator) {
        if (n == 0) return accumulator;
        if (n % 2 == 1) accumulator = accumulator * x;
        return power_tail(x * x, n / 2, accumulator);
    }

    static String my_pow(double x, long n, Validator validate) {
        if (!validate.validate(x, n)) return "INVALID";
        double result;
        if (n < 0) {
            x = 1 / x;
            if (n == Long.MIN_VALUE) {
                result = power_tail(x, Long.MAX_VALUE, x);
            } else {
                result = power_tail(x, -n, 1.0);
            }
        } else {
            result = power_tail(x, n, 1.0);
        }
        return String.format(Locale.US, "%.5f", result);
    }

    public static void main(String[] args) {
```

```
31          Scanner sc = new Scanner(System.in);
32          sc.useLocale(Locale.US);
33          double x = sc.nextDouble();
34          long n = sc.nextLong();
35          Validator v = (a, b) -> !(a == 0 && b < 0);
36          System.out.println(my_pow(x, n, v));
37      }
38  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner and Locale.
3	`public class Main {`	The class that holds the program.
4	`    interface Validator {`	Interface with one method: the Java way to pass a function (callback) as a parameter.
5	`        boolean validate(double x, long n);`	Returns true if computing x^n is valid.
6	`    }`	End of the interface.
8	`    static double power_tail(double x, long n, double accumulator) {`	Tail-recursive fast power. The accumulator carries the answer built so far.
9	`        if (n == 0) return accumulator;`	Base case: when the exponent reaches 0, the accumulator is the answer.
10	`        if (n % 2 == 1) accumulator = accumulator * x;`	If the exponent is odd, multiply one extra x into the accumulator.
11	`        return power_tail(x * x, n / 2, accumulator);`	Square the base and halve the exponent. The recursive call is the last action (tail call), so only O(log n) calls are made.
12	`    }`	End of power_tail.
14	`    static String my_pow(double x, long n, Validator validate) {`	Wrapper method; it receives the validator (callback) as a parameter.
15	`        if (!validate.validate(x, n)) return "INVALID";`	Call the callback; if the input is invalid, return INVALID.
16	`        double result;`	Will hold x^n.
17	`        if (n < 0) {`	For a negative exponent, x^(-n) = (1/x)^n.
18	`            x = 1 / x;`	Replace the base by its reciprocal.
19	`            if (n == Long.MIN_VALUE) {`	Special case: -n would overflow for the smallest long value.
20	`                result = power_tail(x, Long.MAX_VALUE, x);`	Use MAX_VALUE and start the accumulator at x for the one missing factor.
21	`            } else {`	Normal negative exponent.
22	`                result = power_tail(x, -n, 1.0);`	Make the exponent positive and start with accumulator 1.0.
23	`            }`	End of the inner if/else.
24	`        } else {`	Exponent is zero or positive.
25	`            result = power_tail(x, n, 1.0);`	Start the recursion with accumulator 1.0.
26	`        }`	End of the outer if/else.
27	`        return String.format(Locale.US, "%.5f", result);`	Format the answer to 5 decimal places (Locale.US keeps the decimal point).
28	`    }`	End of my_pow.
30	`    public static void main(String[] args) {`	The program starts running here.
31	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.

Line	Code	Explanation
32	`sc.useLocale(Locale.US);`	Make Scanner read decimals with a dot.
33	`double x = sc.nextDouble();`	Read the base.
34	`long n = sc.nextLong();`	Read the exponent.
35	`Validator v = (a, b) -> !(a == 0 && b < 0);`	Lambda callback: invalid only when the base is 0 and the exponent is negative.
36	`System.out.println(my_pow(x, n, v));`	Pass the callback and print the result.
37	`}`	End of main.
38	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: x = 2.0, n = -10. The validator lambda returns true (x is not 0). Since n < 0, x becomes 1/2 = 0.5 and n becomes 10. Recursion starts with accumulator = 1.0.

Call	x	n	n odd?	accumulator after	Next call
1	0.5	10	No	1.0	power_tail(0.25, 5, 1.0)
2	0.25	5	Yes	1.0 * 0.25 = 0.25	power_tail(0.0625, 2, 0.25)
3	0.0625	2	No	0.25	power_tail(0.00390625, 1, 0.25)
4	0.00390625	1	Yes	0.25 * 0.00390625 = 0.0009765625	power_tail(x*x, 0, 0.0009765625)
5	(tiny)	0	-	0.0009765625	Base case: return accumulator

Result

Result = 0.0009765625, formatted with %.5f gives 0.00098. Output: 0.00098

Invalid example: input 0 -1 makes the validator return false, so the program prints INVALID.

Question 2.2

Recursive Subset Generation via Pass-by-Reference

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a set of N distinct positive integers, write a recursive C program to generate and print all possible unique subsets (the power set).

Constraints / Requirements

- Memory allocation for storing current subset paths must be handled dynamically and passed down the call stack using Pass-by-Reference (pointers).
- Do not use global variables.

Input Format

First line: Integer N (1 <= N <= 10).
Second line: N space-separated distinct integers.

Output Format

Print each subset on a new line in lexicographical format. Print [] for the empty set.

Sample Input	Sample Output
3 1 2 3	[] [1] [1, 2] [1, 2, 3] [1, 3] [2] [2, 3] [3]

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      static void generate(int[] arr, int start, ArrayList<Integer> current) {
 5          System.out.println(current);
 6          for (int i = start; i < arr.length; i++) {
 7              current.add(arr[i]);
 8              generate(arr, i + 1, current);
 9              current.remove(current.size() - 1);
10          }
11      }
12
13      public static void main(String[] args) {
14          Scanner sc = new Scanner(System.in);
15          int n = sc.nextInt();
16          int[] arr = new int[n];
17          for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
18          Arrays.sort(arr);
19          generate(arr, 0, new ArrayList<Integer>());
20      }
21  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner, ArrayList and Arrays.

Line	Code	Explanation
3	`public class Main {`	The class that holds the program.
4	`static void generate(int[] arr, int start, ArrayList<Integer> current) {`	Recursive method. current is an object reference, so every call shares the same list (pass by reference).
5	`System.out.println(current);`	Each call prints one subset: the current list.
6	`for (int i = start; i < arr.length; i++) {`	Try every element from position start onwards.
7	`current.add(arr[i]);`	Choose arr[i]: add it to the subset.
8	`generate(arr, i + 1, current);`	Recurse to extend the subset using only the later elements.
9	`current.remove(current.size() - 1);`	Backtrack: remove the last element so the next choice starts clean.
10	`}`	End of the loop.
11	`}`	End of generate.
13	`public static void main(String[] args) {`	The program starts running here.
14	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
15	`int n = sc.nextInt();`	Read N, the number of elements.
16	`int[] arr = new int[n];`	Array to hold the N integers.
17	`for (int i = 0; i < n; i++) arr[i] = sc.nextInt();`	Read the N integers.
18	`Arrays.sort(arr);`	Sort so subsets are printed in lexicographical order.
19	`generate(arr, 0, new ArrayList<Integer>());`	Start with an empty subset and index 0.
20	`}`	End of main.
21	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: arr = [1, 2, 3]. Every call to generate() prints the current list first, then tries to add each later element one by one (choose, recurse, undo).

#	Call generate(start, current)	Printed	What happens next
1	generate(0, [])	[]	choose 1, recurse
2	generate(1, [1])	[1]	choose 2, recurse
3	generate(2, [1, 2])	[1, 2]	choose 3, recurse
4	generate(3, [1, 2, 3])	[1, 2, 3]	loop is empty; return; pop 3, pop 2; now choose 3 after [1]
5	generate(3, [1, 3])	[1, 3]	return; pop 3, pop 1; now choose 2 after []
6	generate(2, [2])	[2]	choose 3, recurse
7	generate(3, [2, 3])	[2, 3]	return; pop 3, pop 2; now choose 3 after []
8	generate(3, [3])	[3]	return; all loops finish

Result

Eight lines are printed in the order shown above (2^3 = 8 subsets), which is the required output.

Session 3: Pointers, Arrays & Strings

Question 3.1

In-Place 2D Matrix Rotation using Pointer Arithmetic

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Write a C program that dynamically allocates a 2D N x N matrix of integers using double pointers (int**). The program must rotate the matrix 90 degrees clockwise in-place using pure pointer arithmetic (avoiding array subscript notation matrix[i][j]).

Constraints / Requirements

- Dynamic allocation using malloc/calloc.
- Perform rotation in O(1) extra space by first transposing using pointers and then reversing each row.
- Free all dynamically allocated memory before program termination.

Input Format

First line contains integer N (1 <= N <= 100).
Next N lines contain N space-separated integers each.

Output Format

Print the N x N rotated matrix.

Sample Input	Sample Output
3 1 2 3 4 5 6 7 8 9	7 4 1 8 5 2 9 6 3

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          Scanner sc = new Scanner(System.in);
 6          int n = sc.nextInt();
 7          int[][] m = new int[n][n];
 8          for (int i = 0; i < n; i++)
 9              for (int j = 0; j < n; j++)
10                  m[i][j] = sc.nextInt();
11
12          for (int i = 0; i < n; i++) {
13              for (int j = i + 1; j < n; j++) {
14                  int t = m[i][j];
15                  m[i][j] = m[j][i];
16                  m[j][i] = t;
17              }
18          }
19
20          for (int i = 0; i < n; i++) {
21              int l = 0, r = n - 1;
22              while (l < r) {
23                  int t = m[i][l];
24                  m[i][l] = m[i][r];
25                  m[i][r] = t;
26                  l++;
27                  r--;
28              }
```

```
        }

        for (int i = 0; i < n; i++) {
            StringBuilder sb = new StringBuilder();
            for (int j = 0; j < n; j++) {
                if (j > 0) sb.append(" ");
                sb.append(m[i][j]);
            }
            System.out.println(sb);
        }
    }
}
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`int n = sc.nextInt();`	Read N, the matrix size.
7	`int[][] m = new int[n][n];`	Create an N x N matrix.
8	`for (int i = 0; i < n; i++)`	Go through each row.
9	`for (int j = 0; j < n; j++)`	Go through each column.
10	`m[i][j] = sc.nextInt();`	Read the element.
12	`for (int i = 0; i < n; i++) {`	Step 1, transpose: go through each row i.
13	`for (int j = i + 1; j < n; j++) {`	Only visit elements above the diagonal so each pair is swapped once.
14	`int t = m[i][j];`	Save m[i][j] in a temporary variable.
15	`m[i][j] = m[j][i];`	Copy the mirror element into m[i][j].
16	`m[j][i] = t;`	Put the saved value into the mirror position.
17	`}`	End of the inner loop.
18	`}`	End of the transpose loop.
20	`for (int i = 0; i < n; i++) {`	Step 2: reverse every row.
21	`int l = 0, r = n - 1;`	Two pointers: left end and right end of the row.
22	`while (l < r) {`	Continue until the pointers meet.
23	`int t = m[i][l];`	Save the left element.
24	`m[i][l] = m[i][r];`	Copy the right element to the left.
25	`m[i][r] = t;`	Put the saved value on the right.
26	`l++;`	Move the left pointer right.
27	`r--;`	Move the right pointer left.
28	`}`	End of the while loop.
29	`}`	End of the reverse loop.
31	`for (int i = 0; i < n; i++) {`	Print the matrix row by row.
32	`StringBuilder sb = new StringBuilder();`	StringBuilder builds the line of text.
33	`for (int j = 0; j < n; j++) {`	Go through the columns.
34	`if (j > 0) sb.append(" ");`	Put a space between numbers.
35	`sb.append(m[i][j]);`	Add the number.
36	`}`	End of the column loop.

Line	Code	Explanation
37	`System.out.println(sb);`	Print the finished row.
38	`}`	End of the print loop.
39	`}`	End of main.
40	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: N = 3 with the matrix below. Step 1 transposes it (swap m[i][j] with m[j][i] for j > i: pairs (0,1), (0,2), (1,2)). Step 2 reverses each row.

Stage	Row 0	Row 1	Row 2
Original	1 2 3	4 5 6	7 8 9
After swapping 2 and 4	1 4 3	2 5 6	7 8 9
After swapping 3 and 7	1 4 7	2 5 6	3 8 9
After swapping 6 and 8 (transpose done)	1 4 7	2 5 8	3 6 9
After reversing each row	7 4 1	8 5 2	9 6 3

Result

```
The matrix is rotated 90 degrees clockwise. Output:
7 4 1
8 5 2
9 6 3
```

Question 3.2

Custom String Compression Engine using Pointer Traversal

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Write a C function char* compressString(const char* str) that takes an input string and returns a dynamically allocated string representing its run-length compression (e.g., "aabccccccaaa" becomes "a2b1c5a3").

Constraints / Requirements

- You cannot use standard string library functions like strlen, strcpy, or strcat.
- All string traversals and manipulations must be executed using pointer incrementing and dereferencing.
- If the compressed string length is not smaller than the original string length, return a duplicate of the original string.

Input Format

A single string containing uppercase and lowercase ASCII letters (max length 1000).

Output Format

The compressed string (or original string if compression offers no length reduction).

Sample Input	Sample Output
aabccccccaaa	a2b1c5a3

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      static String compressString(String str) {
 5          char[] s = str.toCharArray();
 6          int len = s.length;
 7          StringBuilder sb = new StringBuilder();
 8          int i = 0;
 9          while (i < len) {
10              int j = i;
11              while (j < len && s[j] == s[i]) j++;
12              sb.append(s[i]);
13              sb.append(j - i);
14              i = j;
15          }
16          if (sb.length() < len) return sb.toString();
17          return str;
18      }
19
20      public static void main(String[] args) {
21          Scanner sc = new Scanner(System.in);
22          String str = sc.hasNext() ? sc.next() : "";
23          System.out.println(compressString(str));
24      }
25  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`static String compressString(String str) {`	Returns the run-length compressed string.

Line	Code	Explanation
5	`char[] s = str.toCharArray();`	Convert the string to a character array to walk through it.
6	`int len = s.length;`	Length of the original string.
7	`StringBuilder sb = new StringBuilder();`	The compressed text is built here.
8	`int i = 0;`	i marks the start of the current run of equal characters.
9	`while (i < len) {`	Repeat until the whole string is processed.
10	`int j = i;`	j will move to the end of the run.
11	`while (j < len && s[j] == s[i]) j++;`	Move j forward while the character stays the same.
12	`sb.append(s[i]);`	Append the character.
13	`sb.append(j - i);`	Append the run length (j - i).
14	`i = j;`	Start the next run where this one ended.
15	`}`	End of the loop.
16	`if (sb.length() < len) return sb.toString();`	If the compressed text is shorter, return it.
17	`return str;`	Otherwise return the original string.
18	`}`	End of compressString.
20	`public static void main(String[] args) {`	The program starts running here.
21	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
22	`String str = sc.hasNext() ? sc.next() : "";`	Read the string (empty string if there is no input).
23	`System.out.println(compressString(str));`	Compress it and print the result.
24	`}`	End of main.
25	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: aabccccccaaa. Characters by index: 0 a, 1 a, 2 b, 3 c, 4 c, 5 c, 6 c, 7 c, 8 a, 9 a, 10 a. Original length n = 11.

Run	i (start)	j (end, exclusive)	Run length j - i	Appended	result so far
1	0	2	2	a2	a2
2	2	3	1	b1	a2b1
3	3	8	5	c5	a2b1c5
4	8	11	3	a3	a2b1c5a3

Result

length(result) = 8, which is less than 11, so the compressed string is returned.
Output: a2b1c5a3

Session 4: Structures, Unions & File Handling

Question 4.1

Student Registry Binary File Indexing & Filtering

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Design a C program for a university evaluation engine:
1. Define a structure Student with attributes: int id, char name[50], and float gpa.
2. Read data for N students and save the structures directly into a binary file named records.db using fwrite().
3. Open records.db, parse the records using fread(), dynamically allocate an array of Student structures for students with a GPA >= 8.5, and print their details.

Input Format

First line: Integer N.
Next N lines: id name gpa (space-separated).

Output Format

Print filtered records of eligible students (GPA >= 8.5), each on a new line formatted as: ID: <id> | Name: <name> | GPA: <gpa>.

Sample Input	Sample Output
3 101 Alice 9.1 102 Bob 7.8 103 Charlie 8.7	ID: 101 | Name: Alice | GPA: 9.10 ID: 103 | Name: Charlie | GPA: 8.70

2 Java Solution

```
 1  import java.io.*;
 2  import java.util.*;
 3
 4  public class Main {
 5      static class Student {
 6          int id;
 7          String name;
 8          float gpa;
 9      }
10
11      public static void main(String[] args) throws Exception {
12          Scanner sc = new Scanner(System.in);
13          sc.useLocale(Locale.US);
14          int n = sc.nextInt();
15
16          FileOutputStream fos = new FileOutputStream("records.db");
17          DataOutputStream out = new DataOutputStream(fos);
18          for (int i = 0; i < n; i++) {
19              out.writeInt(sc.nextInt());
20              out.writeUTF(sc.next());
21              out.writeFloat(sc.nextFloat());
22          }
23          out.close();
24
25          FileInputStream fis = new FileInputStream("records.db");
26          DataInputStream in = new DataInputStream(fis);
27          ArrayList<Student> list = new ArrayList<>();
28          for (int i = 0; i < n; i++) {
```

```
29              Student s = new Student();
30              s.id = in.readInt();
31              s.name = in.readUTF();
32              s.gpa = in.readFloat();
33              if (s.gpa >= 8.5f) list.add(s);
34          }
35          in.close();
36
37          for (Student s : list) {
38              String fmt = "ID: %d | Name: %s | GPA: %.2f";
39              System.out.println(String.format(Locale.US, fmt, s.id, s.name, s.gpa));
40          }
41      }
42  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.io.*;`	Import file classes: DataOutputStream, DataInputStream, FileOutputStream, FileInputStream.
2	`import java.util.*;`	Import Scanner, ArrayList and Locale.
4	`public class Main {`	The class that holds the program.
5	`    static class Student {`	Java version of the C structure Student.
6	`        int id;`	Student ID.
7	`        String name;`	Student name.
8	`        float gpa;`	Student GPA.
9	`    }`	End of the Student class.
11	`    public static void main(String[] args) throws Exception {`	"throws Exception" lets us use file operations without try/catch.
12	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
13	`        sc.useLocale(Locale.US);`	Make Scanner read decimals with a dot.
14	`        int n = sc.nextInt();`	Read N, the number of students.
16	`        FileOutputStream fos = new FileOutputStream("records.db");`	Open (create) the file records.db for writing.
17	`        DataOutputStream out = new DataOutputStream(fos);`	Wrap it so we can write ints, strings and floats as binary data.
18	`        for (int i = 0; i < n; i++) {`	Repeat for each student.
19	`            out.writeInt(sc.nextInt());`	Write the ID as 4 bytes.
20	`            out.writeUTF(sc.next());`	Write the name.
21	`            out.writeFloat(sc.nextFloat());`	Write the GPA as 4 bytes.
22	`        }`	End of the writing loop.
23	`        out.close();`	Close the file so everything is saved.
25	`        FileInputStream fis = new FileInputStream("records.db");`	Open the file records.db for reading.
26	`        DataInputStream in = new DataInputStream(fis);`	Wrap it so we can read ints, strings and floats from binary data.
27	`        ArrayList<Student> list = new ArrayList<>();`	Dynamic list for students with GPA >= 8.5.
28	`        for (int i = 0; i < n; i++) {`	Read N records.
29	`            Student s = new Student();`	Create a Student object.
30	`            s.id = in.readInt();`	Read the ID (same order as written).
31	`            s.name = in.readUTF();`	Read the name.

Line	Code	Explanation
32	`s.gpa = in.readFloat();`	Read the GPA.
33	`if (s.gpa >= 8.5f) list.add(s);`	Keep only students whose GPA is at least 8.5.
34	`}`	End of the reading loop.
35	`in.close();`	Close the file.
37	`for (Student s : list) {`	Print every eligible student.
38	`String fmt = "ID: %d | Name: %s | GPA: %.2f";`	The required output format; %.2f shows 2 decimals.
39	`System.out.println(String.format(Locale.US, fmt, s.id, s.name, s.gpa));`	Fill in the values and print the line.
40	`}`	End of the print loop.
41	`}`	End of main.
42	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: N = 3. Each student is written to records.db with writeInt, writeUTF and writeFloat. Then the file is read back with readInt, readUTF and readFloat in the same order.

Student	Written to records.db	GPA >= 8.5 ?	Printed
101 Alice 9.1	101, "Alice", 9.1	9.1 >= 8.5, Yes	ID: 101 | Name: Alice | GPA: 9.10
102 Bob 7.8	102, "Bob", 7.8	7.8 >= 8.5, No	(skipped)
103 Charlie 8.7	103, "Charlie", 8.7	8.7 >= 8.5, Yes	ID: 103 | Name: Charlie | GPA: 8.70

Result

```
Output:
ID: 101 | Name: Alice | GPA: 9.10
ID: 103 | Name: Charlie | GPA: 8.70
```

Question 4.2

Heterogeneous Sensor Log Parser using Tagged Unions

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

An IoT device logs mixed sensor readings into a file sensor_log.txt. Each log entry consists of a type flag (1 for Temperature, 2 for Status, 3 for Count).
Write a C program to:
1. Define a tagged union structure SensorReading (with enum for type and union for float, char[20], and int).
2. Parse the formatted input file line-by-line using file functions (fopen, fscanf, fclose).
3. Store entries in a dynamically allocated array of SensorReading structures and output formatted logs based on their union payload type.

Input Format

A sequence of inputs where each line represents a record: <TypeFlag> <Value>.
The last line contains -1 indicating End-Of-File / stop reading.

Output Format

Print each record with its mapped tag: [TEMP] ..., [STATUS] ..., [COUNT] ...

Sample Input	Sample Output
1 36.5 2 OK 3 142 -1	[TEMP] 36.50 C [STATUS] OK [COUNT] 142

2 Java Solution

```
 1  import java.io.*;
 2  import java.util.*;
 3
 4  public class Main {
 5      enum Type { TEMP, STATUS, COUNT }
 6
 7      static class SensorReading {
 8          Type type;
 9          float temp;
10          String status;
11          int count;
12      }
13
14      public static void main(String[] args) throws Exception {
15          File f = new File("sensor_log.txt");
16          Scanner sc;
17          if (f.exists()) sc = new Scanner(f);
18          else sc = new Scanner(System.in);
19          sc.useLocale(Locale.US);
20
21          ArrayList<SensorReading> list = new ArrayList<>();
22          while (sc.hasNextInt()) {
23              int flag = sc.nextInt();
24              if (flag == -1) break;
25              SensorReading r = new SensorReading();
26              if (flag == 1) {
27                  r.type = Type.TEMP;
28                  r.temp = sc.nextFloat();
29              } else if (flag == 2) {
30                  r.type = Type.STATUS;
```

```
31                 r.status = sc.next();
32             } else {
33                 r.type = Type.COUNT;
34                 r.count = sc.nextInt();
35             }
36             list.add(r);
37         }
38         sc.close();
39 
40         for (SensorReading r : list) {
41             if (r.type == Type.TEMP) {
42                 System.out.println(String.format(Locale.US, "[TEMP] %.2f C", r.temp));
43             }
44             else if (r.type == Type.STATUS) System.out.println("[STATUS] " + r.status);
45             else System.out.println("[COUNT] " + r.count);
46         }
47     }
48 }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.io.*;`	Import File.
2	`import java.util.*;`	Import Scanner, ArrayList and Locale.
4	`public class Main {`	The class that holds the program.
5	`enum Type { TEMP, STATUS, COUNT }`	Enum: the tag that tells which kind of reading it is.
7	`static class SensorReading {`	Tagged union: a tag plus one value that is used depending on the tag.
8	`Type type;`	The tag (TEMP, STATUS or COUNT).
9	`float temp;`	Used when type is TEMP.
10	`String status;`	Used when type is STATUS.
11	`int count;`	Used when type is COUNT.
12	`}`	End of the SensorReading class.
14	`public static void main(String[] args) throws Exception {`	"throws Exception" lets us open files without try/catch.
15	`File f = new File("sensor_log.txt");`	Represents the file sensor_log.txt.
16	`Scanner sc;`	Scanner that will read the log.
17	`if (f.exists()) sc = new Scanner(f);`	If the file exists, read from the file (like fopen).
18	`else sc = new Scanner(System.in);`	Otherwise read from the keyboard.
19	`sc.useLocale(Locale.US);`	Make Scanner read decimals with a dot.
21	`ArrayList<SensorReading> list = new ArrayList<>();`	Dynamic list that stores all readings.
22	`while (sc.hasNextInt()) {`	Continue while the next token is an integer (the flag).
23	`int flag = sc.nextInt();`	Read the type flag.
24	`if (flag == -1) break;`	-1 marks the end of the log, so stop reading.
25	`SensorReading r = new SensorReading();`	Create a new reading.
26	`if (flag == 1) {`	Flag 1: temperature.
27	`r.type = Type.TEMP;`	Set the tag.
28	`r.temp = sc.nextFloat();`	Read the temperature value.
29	`} else if (flag == 2) {`	Flag 2: status.
30	`r.type = Type.STATUS;`	Set the tag.
31	`r.status = sc.next();`	Read the status text.

Line	Code	Explanation
32	`} else {`	Flag 3: count.
33	`r.type = Type.COUNT;`	Set the tag.
34	`r.count = sc.nextInt();`	Read the count value.
35	`}`	End of the if/else chain.
36	`list.add(r);`	Store the reading in the list.
37	`}`	End of the reading loop.
38	`sc.close();`	Close the input (like fclose).
40	`for (SensorReading r : list) {`	Print every stored reading.
41	`if (r.type == Type.TEMP) {`	Temperature reading.
42	`System.out.println(String.format(Locale.US, "[TEMP] %.2f C", r.temp));`	Print with 2 decimals and the unit C.
43	`}`	End of the temperature case.
44	`else if (r.type == Type.STATUS) System.out.println("[STATUS] " + r.status);`	Status reading: print the tag and the text.
45	`else System.out.println("[COUNT] " + r.count);`	Count with its value.
46	`}`	End of the print loop.
47	`}`	End of main.
48	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input lines: 1 36.5 / 2 OK / 3 142 / -1. The program reads the flag first, then the value that goes with it.

Flag read	Value read	Action	list after
1	36.5	type = TEMP, temp = 36.5, add to list	[TEMP 36.5]
2	OK	type = STATUS, status = "OK", add to list	[TEMP 36.5, STATUS OK]
3	142	type = COUNT, count = 142, add to list	[TEMP 36.5, STATUS OK, COUNT 142]
-1	-	break: stop reading	unchanged

Result

```
Printing the three readings gives:
[TEMP] 36.50 C
[STATUS] OK
[COUNT] 142
```

Session 5: Linked Lists in C

Question 5.1

Reverse Nodes in k-Group

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a singly linked list, reverse the nodes of the linked list k at a time and return its modified list. If the number of nodes is not a multiple of k, then left-out nodes in the end should remain as it is.

Constraints / Requirements

- Time Complexity: O(N)
- Space Complexity: O(1) auxiliary space (in-place modification).
- Only modify the pointers, do not swap data values.

Input Format

First line: Integer N (number of nodes) and Integer K.
Second line: N space-separated integers representing the linked list.

Output Format

Space-separated integers representing the modified linked list.

Sample Input	Sample Output
5 2 1 2 3 4 5	2 1 4 3 5

2 Java Solution

```
import java.util.*;

public class Main {
    static class Node {
        int data;
        Node next;
        Node(int d) { data = d; }
    }

    static Node reverseKGroup(Node head, int k) {
        Node dummy = new Node(0);
        dummy.next = head;
        Node prev = dummy;
        while (true) {
            Node end = prev;
            for (int i = 0; i < k && end != null; i++) end = end.next;
            if (end == null) break;

            Node start = prev.next;
            Node after = end.next;
            Node p = after, cur = start;
            while (cur != after) {
                Node t = cur.next;
                cur.next = p;
                p = cur;
                cur = t;
            }
            prev.next = end;
            prev = start;
        }
        return dummy.next;
```

```
32      }
33
34      public static void main(String[] args) {
35          Scanner sc = new Scanner(System.in);
36          int n = sc.nextInt();
37          int k = sc.nextInt();
38          Node head = null, tail = null;
39          for (int i = 0; i < n; i++) {
40              Node nd = new Node(sc.nextInt());
41              if (head == null) head = tail = nd;
42              else { tail.next = nd; tail = nd; }
43          }
44          head = reverseKGroup(head, k);
45          StringBuilder sb = new StringBuilder();
46          for (Node c = head; c != null; c = c.next) {
47              if (sb.length() > 0) sb.append(" ");
48              sb.append(c.data);
49          }
50          System.out.println(sb);
51      }
52  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`    static class Node {`	One node of the singly linked list.
5	`        int data;`	The value stored in the node.
6	`        Node next;`	Reference to the next node (null means the end).
7	`        Node(int d) { data = d; }`	Constructor: store the value.
8	`    }`	End of the Node class.
10	`    static Node reverseKGroup(Node head, int k) {`	Reverses the list k nodes at a time.
11	`        Node dummy = new Node(0);`	Dummy node placed before the head, to simplify edge cases.
12	`        dummy.next = head;`	Link the dummy node to the real head.
13	`        Node prev = dummy;`	prev is the node just before the group being reversed.
14	`        while (true) {`	Process group after group.
15	`            Node end = prev;`	Start at prev and walk k steps to find the last node of the group.
16	`            for (int i = 0; i < k && end != null; i++) end = end.next;`	Move forward k times (stop early if the list ends).
17	`            if (end == null) break;`	Fewer than k nodes remain, so leave them as they are and finish.
19	`            Node start = prev.next;`	First node of the group.
20	`            Node after = end.next;`	First node after the group.
21	`            Node p = after, cur = start;`	p is what the reversed part will point to; cur walks through the group.
22	`            while (cur != after) {`	Reverse the pointers of all k nodes.
23	`                Node t = cur.next;`	Save the next node before changing the link.
24	`                cur.next = p;`	Point the current node backwards.
25	`                p = cur;`	p moves forward to the current node.
26	`                cur = t;`	Move to the saved next node.
27	`            }`	End of the reversing loop.

Line	Code	Explanation
28	`prev.next = end;`	The node before the group now points to the new first node (old last).
29	`prev = start;`	The old first node is now the group's tail, so it becomes the new prev.
30	`}`	End of the group loop.
31	`return dummy.next;`	Return the real head of the modified list.
32	`}`	End of reverseKGroup.
34	`public static void main(String[] args) {`	The program starts running here.
35	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
36	`int n = sc.nextInt();`	Number of nodes.
37	`int k = sc.nextInt();`	Group size.
38	`Node head = null, tail = null;`	The list is empty at first.
39	`for (int i = 0; i < n; i++) {`	Create N nodes.
40	`Node nd = new Node(sc.nextInt());`	Make a node with the next value.
41	`if (head == null) head = tail = nd;`	First node: it is both the head and the tail.
42	`else { tail.next = nd; tail = nd; }`	Otherwise attach it after the tail and make it the new tail.
43	`}`	End of the loop.
44	`head = reverseKGroup(head, k);`	Reverse the list in groups of k.
45	`StringBuilder sb = new StringBuilder();`	Builds the output text.
46	`for (Node c = head; c != null; c = c.next) {`	Walk through the list.
47	`if (sb.length() > 0) sb.append(" ");`	Put a space between values.
48	`sb.append(c.data);`	Add the node's value.
49	`}`	End of the loop.
50	`System.out.println(sb);`	Print the modified list.
51	`}`	End of main.
52	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: N = 5, k = 2, list 1 -> 2 -> 3 -> 4 -> 5. A dummy node sits before 1 and prev starts at the dummy.

Round	Group found	Pointer work	List after
1	1, 2 (start = 1, end = 2, after = 3)	cur = 1: 1.next = 3. cur = 2: 2.next = 1. Then dummy.next = 2, prev = node 1	2 -> 1 -> 3 -> 4 -> 5
2	3, 4 (start = 3, end = 4, after = 5)	cur = 3: 3.next = 5. cur = 4: 4.next = 3. Then node 1.next = 4, prev = node 3	2 -> 1 -> 4 -> 3 -> 5
3	Only node 5 is left (fewer than k = 2 nodes)	end becomes null, so break	2 -> 1 -> 4 -> 3 -> 5

Result

`The leftover node 5 stays as it is. Output: 2 1 4 3 5`

Question 5.2

Rotate Doubly Linked List

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a doubly linked list, rotate the linked list counter-clockwise by P nodes. Here P is a given positive integer and is smaller than the count of nodes in linked list.

Constraints / Requirements

- Time Complexity: O(N)
- Space Complexity: O(1)
- Ensure the previous (prev) and next (next) pointers are properly updated for all affected nodes.

Input Format

First line: Integer N (number of nodes) and Integer P (rotation count).
Second line: N space-separated integers.

Output Format

Space-separated integers of the rotated doubly linked list.

Sample Input	Sample Output
6 2 1 2 3 4 5 6	3 4 5 6 1 2

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      static class Node {
 5          int data;
 6          Node prev, next;
 7          Node(int d) { data = d; }
 8      }
 9
10      public static void main(String[] args) {
11          Scanner sc = new Scanner(System.in);
12          int n = sc.nextInt();
13          int p = sc.nextInt();
14          Node head = null, tail = null;
15          for (int i = 0; i < n; i++) {
16              Node nd = new Node(sc.nextInt());
17              if (head == null) head = tail = nd;
18              else { tail.next = nd; nd.prev = tail; tail = nd; }
19          }
20          p = p % n;
21          if (p != 0) {
22              Node nth = head;
23              for (int i = 1; i < p; i++) nth = nth.next;
24              Node newHead = nth.next;
25              tail.next = head;
26              head.prev = tail;
27              newHead.prev = null;
28              nth.next = null;
29              head = newHead;
30          }
31          StringBuilder sb = new StringBuilder();
32          for (Node c = head; c != null; c = c.next) {
33              if (sb.length() > 0) sb.append(" ");
34              sb.append(c.data);
35          }
```

```
36          System.out.println(sb);
37      }
38  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`static class Node {`	One node of the doubly linked list.
5	`int data;`	The value stored in the node.
6	`Node prev, next;`	References to the previous and next nodes.
7	`Node(int d) { data = d; }`	Constructor: store the value.
8	`}`	End of the Node class.
10	`public static void main(String[] args) {`	The program starts running here.
11	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
12	`int n = sc.nextInt();`	Number of nodes.
13	`int p = sc.nextInt();`	Number of nodes to rotate by.
14	`Node head = null, tail = null;`	The list is empty at first.
15	`for (int i = 0; i < n; i++) {`	Create N nodes.
16	`Node nd = new Node(sc.nextInt());`	Make a node with the next value.
17	`if (head == null) head = tail = nd;`	First node: it is both the head and the tail.
18	`else { tail.next = nd; nd.prev = tail; tail = nd; }`	Link the old tail and the new node in both directions.
19	`}`	End of the loop.
20	`p = p % n;`	Rotating by n or more is the same as rotating by p mod n.
21	`if (p != 0) {`	Rotate only when needed.
22	`Node nth = head;`	Start at the head to find the p-th node.
23	`for (int i = 1; i < p; i++) nth = nth.next;`	Move p - 1 steps forward.
24	`Node newHead = nth.next;`	The node after the p-th node becomes the new head.
25	`tail.next = head;`	Old tail points forward to the old head (the list becomes circular).
26	`head.prev = tail;`	Old head points back to the old tail.
27	`newHead.prev = null;`	The new head has no previous node.
28	`nth.next = null;`	The p-th node becomes the new tail, so it has no next node.
29	`head = newHead;`	Update the head.
30	`}`	End of the rotation.
31	`StringBuilder sb = new StringBuilder();`	Builds the output text.
32	`for (Node c = head; c != null; c = c.next) {`	Walk through the list.
33	`if (sb.length() > 0) sb.append(" ");`	Put a space between values.
34	`sb.append(c.data);`	Add the node's value.
35	`}`	End of the loop.
36	`System.out.println(sb);`	Print the rotated list.
37	`}`	End of main.

Line	Code	Explanation
38	}	End of class Main.

4 Dry Run (Sample Test Case)

Input: N = 6, P = 2, list 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6. p % n = 2.

Step	Action	Result
1	Find nth: start at head (1) and move p - 1 = 1 step	nth = node 2
2	new_head = nth.next	new_head = node 3
3	tail.next = head and head.prev = tail	node 6 <-> node 1 (list is circular)
4	new_head.prev = null and nth.next = null	list is cut between 2 and 3
5	head = new_head	head = node 3

Result

Reading from the new head: 3 4 5 6 1 2. Output: 3 4 5 6 1 2

Session 6: STL - Stacks, Queues & Priority Queues

Question 6.1

Next Greater Element

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given an array of integers, find the Next Greater Element (NGE) for every element. The Next greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider next greater element as -1.

Constraints / Requirements

- Must be solved using a Stack data structure.
- Time Complexity: O(N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers.

Output Format

N space-separated integers representing the NGE for each element.

Sample Input	Sample Output
4 4 5 2 25	5 25 25 -1

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          Scanner sc = new Scanner(System.in);
 6          int n = sc.nextInt();
 7          int[] a = new int[n];
 8          for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 9
10          int[] res = new int[n];
11          Arrays.fill(res, -1);
12          Deque<Integer> stack = new ArrayDeque<>();
13          for (int i = 0; i < n; i++) {
14              while (!stack.isEmpty() && a[stack.peek()] < a[i]) {
15                  res[stack.pop()] = a[i];
16              }
17              stack.push(i);
18          }
19          StringBuilder sb = new StringBuilder();
20          for (int i = 0; i < n; i++) {
21              if (i > 0) sb.append(" ");
22              sb.append(res[i]);
23          }
24          System.out.println(sb);
25      }
26  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner, Deque and ArrayDeque.
3	`public class Main {`	The class that holds the program.
4	`    public static void main(String[] args) {`	The program starts running here.
5	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`        int n = sc.nextInt();`	Number of elements.
7	`        int[] a = new int[n];`	Array for the elements.
8	`        for (int i = 0; i < n; i++) a[i] = sc.nextInt();`	Read the elements.
10	`        int[] res = new int[n];`	Answer array.
11	`        Arrays.fill(res, -1);`	Fill with -1 (meaning: no greater element found yet).
12	`        Deque<Integer> stack = new ArrayDeque<>();`	Stack of indices whose next greater element is not found yet.
13	`        for (int i = 0; i < n; i++) {`	Go through the array from left to right.
14	`            while (!stack.isEmpty() && a[stack.peek()] < a[i]) {`	While the stack is not empty and its top element is smaller than a[i]...
15	`                res[stack.pop()] = a[i];`	...a[i] is the next greater element of the top, so record it and pop.
16	`            }`	End of the while loop.
17	`            stack.push(i);`	Push the current index; its answer is not known yet.
18	`        }`	End of the for loop.
19	`        StringBuilder sb = new StringBuilder();`	Builds the output text.
20	`        for (int i = 0; i < n; i++) {`	Go through the answers.
21	`            if (i > 0) sb.append(" ");`	Put a space between values.
22	`            sb.append(res[i]);`	Add the answer.
23	`        }`	End of the loop.
24	`        System.out.println(sb);`	Print all answers.
25	`    }`	End of main.
26	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: a = [4, 5, 2, 25]. res = [-1, -1, -1, -1]. The stack stores indices.

i	a[i]	Pops (stack top < a[i])	Stack after push	res after
0	4	none (stack empty)	[0]	[-1, -1, -1, -1]
1	5	4 < 5: pop index 0, res[0] = 5	[1]	[5, -1, -1, -1]
2	2	5 < 2 is false: no pop	[1, 2]	[5, -1, -1, -1]
3	25	2 < 25: pop index 2, res[2] = 25; 5 < 25: pop index 1, res[1] = 25	[3]	[5, 25, 25, -1]

Result

Index 3 stays on the stack, so its answer remains -1. Output: 5 25 25 -1

Question 6.2

Minimum Cost of Ropes

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

There are given N ropes of different lengths, we need to connect these ropes into one rope. The cost to connect two ropes is equal to sum of their lengths. The task is to connect the ropes with minimum cost.

Constraints / Requirements

- Must be solved using a Priority Queue (Min-Heap).
- Time Complexity: O(N log N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers representing rope lengths.

Output Format

A single integer representing the minimum cost.

Sample Input	Sample Output
4 4 3 2 6	29

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          Scanner sc = new Scanner(System.in);
 6          int n = sc.nextInt();
 7          PriorityQueue<Long> pq = new PriorityQueue<>();
 8          for (int i = 0; i < n; i++) pq.add(sc.nextLong());
 9
10          long cost = 0;
11          while (pq.size() > 1) {
12              long a = pq.poll();
13              long b = pq.poll();
14              cost += a + b;
15              pq.add(a + b);
16          }
17          System.out.println(cost);
18      }
19  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner and PriorityQueue.
3	`public class Main {`	The class that holds the program.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`int n = sc.nextInt();`	Number of ropes.
7	`PriorityQueue<Long> pq = new PriorityQueue<>();`	A min-heap: the smallest element is always removed first.

Line	Code	Explanation
8	`for (int i = 0; i < n; i++) pq.add(sc.nextLong());`	Add every rope length to the heap.
10	`long cost = 0;`	Total cost so far.
11	`while (pq.size() > 1) {`	Continue until only one rope is left.
12	`long a = pq.poll();`	Take the shortest rope.
13	`long b = pq.poll();`	Take the next shortest rope.
14	`cost += a + b;`	Joining them costs the sum of their lengths.
15	`pq.add(a + b);`	Put the joined rope back in the heap.
16	`}`	End of the loop.
17	`System.out.println(cost);`	Print the minimum total cost.
18	`}`	End of main.
19	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: ropes = 4 3 2 6. The min-heap always gives the two shortest ropes.

Step	Heap before	Pop a, b	Cost of joining	Total cost	Heap after
1	[2, 3, 4, 6]	2, 3	2 + 3 = 5	5	[4, 5, 6]
2	[4, 5, 6]	4, 5	4 + 5 = 9	14	[6, 9]
3	[6, 9]	6, 9	6 + 9 = 15	29	[15]

Result

Only one rope is left, so the loop ends. Output: 29

Session 7: Linked Lists, Binary Trees, Stacks & Queues

Question 7.1

Zigzag Level Order Traversal

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Constraints / Requirements

- Implement using a Queue/Deque or two Stacks.
- Time Complexity: O(N)
- Space Complexity: O(N)

Input Format

A single line containing the level-order representation of the tree (use -1 for NULL nodes).

Output Format

Space-separated integers representing the zigzag traversal.

Sample Input	Sample Output
3 9 20 -1 -1 15 7	3 20 9 15 7

2 Java Solution

```
1  import java.util.*;
2
3  public class Main {
4      static class Node {
5          int val;
6          Node left, right;
7          Node(int v) { val = v; }
8      }
9
10     public static void main(String[] args) {
11         Scanner sc = new Scanner(System.in);
12         ArrayList<Integer> a = new ArrayList<>();
13         while (sc.hasNextInt()) a.add(sc.nextInt());
14
15         if (a.isEmpty() || a.get(0) == -1) {
16             System.out.println();
17             return;
18         }
19         Node root = new Node(a.get(0));
20         Queue<Node> q = new LinkedList<>();
21         q.add(root);
22         int i = 1;
23         while (!q.isEmpty() && i < a.size()) {
24             Node cur = q.poll();
25             if (i < a.size()) {
26                 int v = a.get(i++);
27                 if (v != -1) {
28                     cur.left = new Node(v);
29                     q.add(cur.left);
30                 }
31             }
32             if (i < a.size()) {
33                 int v = a.get(i++);
34                 if (v != -1) {
```

```
                    cur.right = new Node(v);
                    q.add(cur.right);
                }
            }
        }

        StringBuilder sb = new StringBuilder();
        q.clear();
        q.add(root);
        boolean leftToRight = true;
        while (!q.isEmpty()) {
            int size = q.size();
            int[] level = new int[size];
            for (int k = 0; k < size; k++) {
                Node cur = q.poll();
                level[k] = cur.val;
                if (cur.left != null) q.add(cur.left);
                if (cur.right != null) q.add(cur.right);
            }
            if (leftToRight) {
                for (int k = 0; k < size; k++) {
                    if (sb.length() > 0) sb.append(" ");
                    sb.append(level[k]);
                }
            } else {
                for (int k = size - 1; k >= 0; k--) {
                    if (sb.length() > 0) sb.append(" ");
                    sb.append(level[k]);
                }
            }
            leftToRight = !leftToRight;
        }
        System.out.println(sb);
    }
}
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner, ArrayList, Queue and LinkedList.
3	`public class Main {`	The class that holds the program.
4	`static class Node {`	One node of the binary tree.
5	`int val;`	The value stored in the node.
6	`Node left, right;`	Left and right children.
7	`Node(int v) { val = v; }`	Constructor: store the value.
8	`}`	End of the Node class.
10	`public static void main(String[] args) {`	The program starts running here.
11	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
12	`ArrayList<Integer> a = new ArrayList<>();`	The level-order values (-1 means NULL).
13	`while (sc.hasNextInt()) a.add(sc.nextInt());`	Read every integer on the line.
15	`if (a.isEmpty() || a.get(0) == -1) {`	If the tree is empty...
16	`System.out.println();`	...print an empty line...
17	`return;`	...and stop the program.
18	`}`	End of the empty-tree check.
19	`Node root = new Node(a.get(0));`	The first value is the root.
20	`Queue<Node> q = new LinkedList<>();`	Queue of nodes whose children are still to be read.
21	`q.add(root);`	Start with the root.

Line	Code	Explanation
22	`        int i = 1;`	Index of the next value to read.
23	`        while (!q.isEmpty() && i < a.size()) {`	Continue while nodes are waiting and values are left.
24	`            Node cur = q.poll();`	Take the next node from the queue.
25	`            if (i < a.size()) {`	If a value is available for the left child...
26	`                int v = a.get(i++);`	...read it and move on...
27	`                if (v != -1) {`	...if it is not NULL...
28	`                    cur.left = new Node(v);`	...create the left child...
29	`                    q.add(cur.left);`	...and queue it.
30	`                }`	End of the left-child check.
31	`            }`	End of the left-child block.
32	`            if (i < a.size()) {`	If a value is available for the right child...
33	`                int v = a.get(i++);`	...read it and move on...
34	`                if (v != -1) {`	...if it is not NULL...
35	`                    cur.right = new Node(v);`	...create the right child...
36	`                    q.add(cur.right);`	...and queue it.
37	`                }`	End of the right-child check.
38	`            }`	End of the right-child block.
39	`        }`	End of the tree-building loop.
41	`        StringBuilder sb = new StringBuilder();`	Builds the zigzag output.
42	`        q.clear();`	Empty the queue: some nodes may still be left in it from the building step.
43	`        q.add(root);`	Restart the queue with the root for the traversal.
44	`        boolean leftToRight = true;`	The first level goes left to right.
45	`        while (!q.isEmpty()) {`	Process the tree level by level.
46	`            int size = q.size();`	The queue holds exactly one level right now.
47	`            int[] level = new int[size];`	Array for the values of this level.
48	`            for (int k = 0; k < size; k++) {`	Take every node of this level.
49	`                Node cur = q.poll();`	Remove the next node.
50	`                level[k] = cur.val;`	Record its value.
51	`                if (cur.left != null) q.add(cur.left);`	Queue the left child for the next level.
52	`                if (cur.right != null) q.add(cur.right);`	Queue the right child for the next level.
53	`            }`	End of the level loop.
54	`            if (leftToRight) {`	On left-to-right levels...
55	`                for (int k = 0; k < size; k++) {`	...go from the first value to the last.
56	`                    if (sb.length() > 0) sb.append(" ");`	Put a space between values.
57	`                    sb.append(level[k]);`	Add the value.
58	`                }`	End of the loop.
59	`            } else {`	On right-to-left levels...
60	`                for (int k = size - 1; k >= 0; k--) {`	...go from the last value to the first.
61	`                    if (sb.length() > 0) sb.append(" ");`	Put a space between values.

Line	Code	Explanation
62	`sb.append(level[k]);`	Add the value.
63	`}`	End of the loop.
64	`}`	End of the direction check.
65	`leftToRight = !leftToRight;`	Flip the direction for the next level.
66	`}`	End of the traversal loop.
67	`System.out.println(sb);`	Print the zigzag order.
68	`}`	End of main.
69	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: 3 9 20 -1 -1 15 7. Building the tree: 3 is the root; its children are 9 and 20; 9 has no children (-1, -1); 20 has children 15 and 7. Then the traversal runs level by level.

Level	Nodes in queue (left to right)	Direction	Added to result	Result so far
0	3	Left to right	3	3
1	9, 20	Right to left (reversed)	20, 9	3 20 9
2	15, 7	Left to right	15, 7	3 20 9 15 7

Result

```
Output: 3 20 9 15 7
```

Question 7.2

Implement Queue using Two Stacks

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Implement a Queue data structure using only two Stacks. The implemented queue should support standard operations: enqueue (push to back), dequeue (remove from front), and peek (view front element).

Constraints / Requirements

- Use standard stack operations only (push, pop, top, empty).
- Amortized Time Complexity for Dequeue: O(1).

Input Format

First line: Integer Q (number of queries).
Next Q lines: Query type 1 x (enqueue x), type 2 (dequeue and print), type 3 (peek and print).

Output Format

Outputs for queries of type 2 and 3 on new lines.

Sample Input	Sample Output
5 1 10 1 20 2 1 30 3	10 20

2 Java Solution

```
import java.util.*;

public class Main {
    static Deque<Integer> inStack = new ArrayDeque<>();
    static Deque<Integer> outStack = new ArrayDeque<>();

    static void shift() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) outStack.push(inStack.pop());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int q = sc.nextInt();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < q; i++) {
            int type = sc.nextInt();
            if (type == 1) {
                inStack.push(sc.nextInt());
            } else if (type == 2) {
                shift();
                if (!outStack.isEmpty()) sb.append(outStack.pop()).append("\n");
            } else {
                shift();
                if (!outStack.isEmpty()) sb.append(outStack.peek()).append("\n");
            }
        }
        System.out.print(sb);
    }
```

```
31  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner, Deque and ArrayDeque.
3	`public class Main {`	The class that holds the program.
4	`    static Deque<Integer> inStack = new ArrayDeque<>();`	Stack that receives new elements (enqueue).
5	`    static Deque<Integer> outStack = new ArrayDeque<>();`	Stack that gives elements out in queue order (dequeue / peek).
7	`    static void shift() {`	Moves elements from inStack to outStack when needed.
8	`        if (outStack.isEmpty()) {`	Only when outStack is empty...
9	`            while (!inStack.isEmpty()) outStack.push(inStack.pop());`	...move everything; this reverses the order so the oldest element ends on top.
10	`        }`	End of the check.
11	`    }`	End of shift.
13	`    public static void main(String[] args) {`	The program starts running here.
14	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
15	`        int q = sc.nextInt();`	Number of queries.
16	`        StringBuilder sb = new StringBuilder();`	Collects the printed answers.
17	`        for (int i = 0; i < q; i++) {`	Process each query.
18	`            int type = sc.nextInt();`	The query type.
19	`            if (type == 1) {`	Type 1: enqueue.
20	`                inStack.push(sc.nextInt());`	Push the value on inStack.
21	`            } else if (type == 2) {`	Type 2: dequeue and print.
22	`                shift();`	Make sure outStack has the front element.
23	`                if (!outStack.isEmpty()) sb.append(outStack.pop()).append("\n");`	Remove and record the front element (only if the queue is not empty).
24	`            } else {`	Type 3: peek and print.
25	`                shift();`	Make sure outStack has the front element.
26	`                if (!outStack.isEmpty()) sb.append(outStack.peek()).append("\n");`	Record the front element without removing it.
27	`            }`	End of the type checks.
28	`        }`	End of the query loop.
29	`        System.out.print(sb);`	Print all recorded answers.
30	`    }`	End of main.
31	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: 5 queries. in_stack receives new elements; out_stack gives the front of the queue.

Query	in_stack	out_stack	Printed
1 10 (enqueue)	[10]	[]	-
1 20 (enqueue)	[10, 20]	[]	-
2 (dequeue)	[]	shift moves 20 then 10: [20, 10]; pop gives 10, leaving [20]	10
1 30 (enqueue)	[30]	[20]	-
3 (peek)	[30]	[20] is not empty, so no shift; top is 20	20

Result

```
Output:
10
20
```

Session 8: String Matching

Question 8.1

Rabin-Karp Pattern Matching

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a text string T and a pattern string P, write a program implementing the Rabin-Karp algorithm to find all starting indices of the pattern in the text.

Constraints / Requirements

- Use a prime modulus to calculate the rolling hash.
- Handle hash collisions properly by verifying string characters.
- Time Complexity: Average O(N+M), Worst O(N*M).

Input Format

First line: Text string T.
Second line: Pattern string P.

Output Format

Space-separated starting indices (0-indexed). Print -1 if no match.

Sample Input	Sample Output
AABAACAADAABAABA AABA	0 9 12

2 Java Solution

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String t = sc.nextLine();
        String p = sc.nextLine();
        int n = t.length(), m = p.length();
        long d = 256, q = 1000000007L;
        StringBuilder sb = new StringBuilder();

        if (m > 0 && m <= n) {
            long h = 1;
            for (int i = 0; i < m - 1; i++) h = (h * d) % q;

            long ph = 0, th = 0;
            for (int i = 0; i < m; i++) {
                ph = (ph * d + p.charAt(i)) % q;
                th = (th * d + t.charAt(i)) % q;
            }
            for (int i = 0; i + m <= n; i++) {
                if (ph == th) {
                    boolean match = true;
                    for (int j = 0; j < m; j++) {
                        if (t.charAt(i + j) != p.charAt(j)) {
                            match = false;
                            break;
                        }
                    }
                    if (match) {
                        if (sb.length() > 0) sb.append(" ");
```

```
32                          sb.append(i);
33                      }
34                  }
35                  if (i + m < n) {
36                      th = (th - t.charAt(i) * h % q + q) % q;
37                      th = (th * d + t.charAt(i + m)) % q;
38                  }
39              }
40          }
41          if (sb.length() == 0) System.out.println(-1);
42          else System.out.println(sb);
43      }
44  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`String t = sc.nextLine();`	Read the text line.
7	`String p = sc.nextLine();`	Read the pattern line.
8	`int n = t.length(), m = p.length();`	Lengths of the text and the pattern.
9	`long d = 256, q = 1000000007L;`	Base of the rolling hash and the prime modulus.
10	`StringBuilder sb = new StringBuilder();`	Collects the match positions.
12	`if (m > 0 && m <= n) {`	Search only when the pattern is not empty and not longer than the text.
13	`long h = 1;`	h will become d^(m-1) mod q.
14	`for (int i = 0; i < m - 1; i++) h = (h * d) % q;`	Multiply by d (m - 1) times.
16	`long ph = 0, th = 0;`	ph = hash of the pattern, th = hash of the current text window.
17	`for (int i = 0; i < m; i++) {`	Hash the pattern and the first window together.
18	`ph = (ph * d + p.charAt(i)) % q;`	Add the next pattern character to the pattern hash.
19	`th = (th * d + t.charAt(i)) % q;`	Add the next text character to the window hash.
20	`}`	End of the loop.
21	`for (int i = 0; i + m <= n; i++) {`	Slide the window over every possible start position.
22	`if (ph == th) {`	If the hashes are equal, the window may match...
23	`boolean match = true;`	...assume a match...
24	`for (int j = 0; j < m; j++) {`	...and verify character by character (handles collisions).
25	`if (t.charAt(i + j) != p.charAt(j)) {`	A character differs...
26	`match = false;`	...so it was only a hash collision...
27	`break;`	...stop checking.
28	`}`	End of the character check.
29	`}`	End of the verify loop.
30	`if (match) {`	All characters matched...
31	`if (sb.length() > 0) sb.append(" ");`	...put a space between indices...
32	`sb.append(i);`	...and record the start index.

Line	Code	Explanation
33	`}`	End of the match block.
34	`}`	End of the hash-equal block.
35	`if (i + m < n) {`	If there is another window to the right...
36	`th = (th - t.charAt(i) * h % q + q) % q;`	...remove the leading character from the hash...
37	`th = (th * d + t.charAt(i + m)) % q;`	...and add the next character (rolling hash).
38	`}`	End of the rolling step.
39	`}`	End of the window loop.
40	`}`	End of the search block.
41	`if (sb.length() == 0) System.out.println(-1);`	No match was found, so print -1.
42	`else System.out.println(sb);`	Otherwise print all start indices.
43	`}`	End of main.
44	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: text = AABAACAADAABAABA (length 16), pattern = AABA (length 4). Pattern hash ph = 94795834. The window hash th is rolled forward one character at a time (remove the first character, add the next). Characters are compared only when the hashes are equal.

Start i	Window	Window hash th	th == ph ?	Characters verified
0	AABA	94795834	Yes	Yes, match
1	ABAA	94861114	No	-
2	BAAC	111572796	No	-
3	AACA	94796090	No	-
4	ACAA	94926650	No	-
5	CAAD	128350013	No	-
6	AADA	94796346	No	-
7	ADAA	94992186	No	-
8	DAAB	145127227	No	-
9	AABA	94795834	Yes	Yes, match
10	ABAA	94861114	No	-
11	BAAB	111572795	No	-
12	AABA	94795834	Yes	Yes, match

Result

Matches found at indices 0, 9, 12. Output: 0 9 12

Question 8.2

Longest Palindromic Substring

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given a string S, find the longest palindromic substring in S. A palindrome is a string that reads the same backwards as forwards.

Constraints / Requirements

- Time Complexity: O(N^2) using expand around center approach or dynamic programming.
- Space Complexity: O(1) or O(N^2).

Input Format

A single string S (length <= 1000).

Output Format

The longest palindromic substring.

Sample Input	Sample Output
babad	bab

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          Scanner sc = new Scanner(System.in);
 6          String s = sc.hasNextLine() ? sc.nextLine() : "";
 7          int n = s.length();
 8          int bestStart = 0, bestLen = 0;
 9
10          for (int c = 0; c < n; c++) {
11              for (int type = 0; type < 2; type++) {
12                  int l = c, r = c + type;
13                  while (l >= 0 && r < n && s.charAt(l) == s.charAt(r)) {
14                      l--;
15                      r++;
16                  }
17                  int len = r - l - 1;
18                  if (len > bestLen) {
19                      bestLen = len;
20                      bestStart = l + 1;
21                  }
22              }
23          }
24          System.out.println(s.substring(bestStart, bestStart + bestLen));
25      }
26  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.

Line	Code	Explanation
6	`        String s = sc.hasNextLine() ? sc.nextLine() : "";`	Read the string (empty if there is no input).
7	`        int n = s.length();`	Length of the string.
8	`        int bestStart = 0, bestLen = 0;`	Start index and length of the longest palindrome so far.
10	`        for (int c = 0; c < n; c++) {`	Treat every position as a possible centre.
11	`            for (int type = 0; type < 2; type++) {`	type 0 tests an odd palindrome; type 1 tests an even palindrome.
12	`                int l = c, r = c + type;`	Left and right pointers start at the centre.
13	`                while (l >= 0 && r < n && s.charAt(l) == s.charAt(r)) {`	Expand outwards while the characters match.
14	`                    l--;`	Move the left pointer left.
15	`                    r++;`	Move the right pointer right.
16	`                }`	End of the expansion.
17	`                int len = r - l - 1;`	Length of the palindrome found (the pointers stopped one step too far).
18	`                if (len > bestLen) {`	If it is longer than the best so far...
19	`                    bestLen = len;`	...save the new length...
20	`                    bestStart = l + 1;`	...and its start index.
21	`                }`	End of the update.
22	`            }`	End of the type loop.
23	`        }`	End of the centre loop.
24	`        System.out.println(s.substring(bestStart, bestStart + bestLen));`	Print the longest palindromic substring.
25	`    }`	End of main.
26	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: babad (indices 0 b, 1 a, 2 b, 3 a, 4 d). For each centre we expand while characters match. length = r - l - 1 after the loop.

Centre c	Type	Final l, r	length	Palindrome	Best so far
0	odd	-1, 1	1	b	len 1, start 0
0	even	0, 1	0	(a, b differ)	unchanged
1	odd	-1, 3	3	bab	len 3, start 0
1	even	1, 2	0	(a, b differ)	unchanged
2	odd	0, 4	3	aba	3 is not > 3, unchanged
2	even	2, 3	0	(b, a differ)	unchanged
3	odd	2, 4	1	a	unchanged
3	even	3, 4	0	(a, d differ)	unchanged
4	odd	3, 5	1	d	unchanged
4	even	4, 5	0	(end of string)	unchanged

Result

best_start = 0 and best_len = 3, so s[0:3] = bab. Output: bab

(Because 'bab' is found first and 'aba' is not longer, 'bab' is printed, matching the question bank.)

Session 9: Performance Analysis of Sorting and Searching

Question 9.1

Search in Rotated Sorted Array

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

There is an integer array nums sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k. Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

Constraints / Requirements

- You must write an algorithm with O(log N) runtime complexity (Binary Search).

Input Format

First line: Integer N.
Second line: N space-separated integers.
Third line: Integer Target.

Output Format

Integer index of the target, or -1.

Sample Input	Sample Output
7 4 5 6 7 0 1 2 0	4

2 Java Solution

```java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int target = sc.nextInt();

        int lo = 0, hi = n - 1, ans = -1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target) { ans = mid; break; }
            if (a[lo] <= a[mid]) {
                if (target >= a[lo] && target < a[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {
                if (target > a[mid] && target <= a[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        System.out.println(ans);
    }
}
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`    public static void main(String[] args) {`	The program starts running here.
5	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`        int n = sc.nextInt();`	Number of elements.
7	`        int[] a = new int[n];`	Array for the rotated array.
8	`        for (int i = 0; i < n; i++) a[i] = sc.nextInt();`	Read the elements.
9	`        int target = sc.nextInt();`	The value to search for.
11	`        int lo = 0, hi = n - 1, ans = -1;`	Search range ends, and the answer (-1 until found).
12	`        while (lo <= hi) {`	Binary search while the range is not empty.
13	`            int mid = lo + (hi - lo) / 2;`	Middle index (written this way to avoid overflow).
14	`            if (a[mid] == target) { ans = mid; break; }`	Found the target: save its index and stop.
15	`            if (a[lo] <= a[mid]) {`	The left half lo..mid is sorted.
16	`                if (target >= a[lo] && target < a[mid]) hi = mid - 1;`	The target lies inside the sorted left half, so search the left half.
17	`                else lo = mid + 1;`	Otherwise search the right half.
18	`            } else {`	Otherwise the right half mid..hi is sorted.
19	`                if (target > a[mid] && target <= a[hi]) lo = mid + 1;`	The target lies inside the sorted right half, so search the right half.
20	`                else hi = mid - 1;`	Otherwise search the left half.
21	`            }`	End of the sorted-half check.
22	`        }`	End of the binary search loop.
23	`        System.out.println(ans);`	Print the index, or -1 if not found.
24	`    }`	End of main.
25	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: a = [4, 5, 6, 7, 0, 1, 2], target = 0. Indices 0 to 6.

lo	hi	mid	a[mid]	Which half is sorted?	Decision
0	6	3	7	a[lo] = 4 <= 7: left half sorted	4 <= 0 < 7 is false, so lo = 4
4	6	5	1	a[lo] = 0 <= 1: left half sorted	0 <= 0 < 1 is true, so hi = 4
4	4	4	0	-	a[mid] == target: ans = 4

Result

```
Output: 4
```

Question 9.2

Inversion Count (Merge Sort)

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given an array of integers. Find the Inversion Count in the array. Two array elements arr[i] and arr[j] form an inversion if arr[i] > arr[j] and i < j.

Constraints / Requirements

- Use a modified Merge Sort approach to count inversions during the merge step.
- Time Complexity: O(N log N)
- Space Complexity: O(N)

Input Format

First line: Integer N.
Second line: N space-separated integers.

Output Format

A single integer representing the inversion count.

Sample Input	Sample Output
5 2 4 1 3 5	3

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      static long mergeCount(int[] a, int[] tmp, int lo, int hi) {
 5          if (lo >= hi) return 0;
 6          int mid = (lo + hi) / 2;
 7          long count = mergeCount(a, tmp, lo, mid) + mergeCount(a, tmp, mid + 1, hi);
 8
 9          int i = lo, j = mid + 1, k = lo;
10          while (i <= mid && j <= hi) {
11              if (a[i] <= a[j]) {
12                  tmp[k++] = a[i++];
13              } else {
14                  tmp[k++] = a[j++];
15                  count += mid - i + 1;
16              }
17          }
18          while (i <= mid) tmp[k++] = a[i++];
19          while (j <= hi) tmp[k++] = a[j++];
20          for (int x = lo; x <= hi; x++) a[x] = tmp[x];
21          return count;
22      }
23
24      public static void main(String[] args) {
25          Scanner sc = new Scanner(System.in);
26          int n = sc.nextInt();
27          int[] a = new int[n];
28          for (int i = 0; i < n; i++) a[i] = sc.nextInt();
29          System.out.println(mergeCount(a, new int[n], 0, n - 1));
30      }
31  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`    static long mergeCount(int[] a, int[] tmp, int lo, int hi) {`	Merge sort on a[lo..hi] that returns the number of inversions.
5	`        if (lo >= hi) return 0;`	One element (or none): already sorted, 0 inversions.
6	`        int mid = (lo + hi) / 2;`	Split point.
7	`        long count = mergeCount(a, tmp, lo, mid) + mergeCount(a, tmp, mid + 1, hi);`	Sort both halves and add up their inversions.
9	`        int i = lo, j = mid + 1, k = lo;`	i walks the left half, j the right half, k the temporary array.
10	`        while (i <= mid && j <= hi) {`	Merge while both halves have elements.
11	`            if (a[i] <= a[j]) {`	The left element is not larger...
12	`                tmp[k++] = a[i++];`	...take it (no inversion).
13	`            } else {`	The right element is smaller...
14	`                tmp[k++] = a[j++];`	...take it...
15	`                count += mid - i + 1;`	...every remaining left element is bigger and comes before it, so each forms an inversion.
16	`            }`	End of the comparison.
17	`        }`	End of the merge loop.
18	`        while (i <= mid) tmp[k++] = a[i++];`	Copy any leftover left elements.
19	`        while (j <= hi) tmp[k++] = a[j++];`	Copy any leftover right elements.
20	`        for (int x = lo; x <= hi; x++) a[x] = tmp[x];`	Copy the merged result back into the array.
21	`        return count;`	Return the inversion count of this part.
22	`    }`	End of mergeCount.
24	`    public static void main(String[] args) {`	The program starts running here.
25	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
26	`        int n = sc.nextInt();`	Number of elements.
27	`        int[] a = new int[n];`	Array for the elements.
28	`        for (int i = 0; i < n; i++) a[i] = sc.nextInt();`	Read the elements.
29	`        System.out.println(mergeCount(a, new int[n], 0, n - 1));`	Count the inversions and print the answer.
30	`    }`	End of main.
31	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: [2, 4, 1, 3, 5] (indices 0 to 4). The first split is mid = (0 + 4) / 2 = 2, giving indices 0..2 and 3..4. An inversion is counted when an element of the right part is taken before the remaining left elements.

Merge	Left part	Right part	What happens	Inversions added
A (idx 0-1)	[2]	[4]	2 <= 4: take 2, then 4	0
B (idx 0-2)	[2, 4]	[1]	2 <= 1 is false: take 1. Pairs (2,1) and (4,1). Then copy 2, 4. Sorted: [1, 2, 4]	mid - i + 1 = 1 - 0 + 1 = 2
C (idx 3-4)	[3]	[5]	3 <= 5: take 3, then 5	0
D (idx 0-4, final)	[1, 2, 4]	[3, 5]	1 <= 3: take 1. 2 <= 3: take 2. 4 <= 3 is false: take 3 (pair (4,3)). 4 <= 5: take 4, then copy 5	mid - i + 1 = 2 - 2 + 1 = 1

Result

`Total inversions = 2 + 1 = 3 (the pairs (2,1), (4,1), (4,3)). Output: 3`

Session 10: Number Theory

Question 10.1

Modular Exponentiation

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given three numbers X, Y, and P, compute (X^Y) % P. This is known as modular exponentiation and is widely used in cryptography.

Constraints / Requirements

- Must be solved using fast exponentiation (Binary Exponentiation).
- Time Complexity: O(log Y)
- Space Complexity: O(1)

Input Format

Three space-separated integers X, Y, and P.

Output Format

A single integer result.

Sample Input	Sample Output
2 3 5	3

2 Java Solution

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          long x = sc.nextLong();
7          long y = sc.nextLong();
8          long p = sc.nextLong();
9
10         long result = 1 % p;
11         x = x % p;
12         while (y > 0) {
13             if (y % 2 == 1) result = (result * x) % p;
14             x = (x * x) % p;
15             y = y / 2;
16         }
17         System.out.println(result);
18     }
19 }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`public static void main(String[] args) {`	The program starts running here.
5	`Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
6	`long x = sc.nextLong();`	Read X.
7	`long y = sc.nextLong();`	Read Y.

Line	Code	Explanation
8	`long p = sc.nextLong();`	Read P.
10	`long result = 1 % p;`	Start with 1 (taken mod p so that p = 1 also works).
11	`x = x % p;`	Reduce the base modulo p.
12	`while (y > 0) {`	Process the exponent bit by bit.
13	`if (y % 2 == 1) result = (result * x) % p;`	If the current bit of y is 1, multiply the result by the current base.
14	`x = (x * x) % p;`	Square the base (mod p).
15	`y = y / 2;`	Shift y right by one bit.
16	`}`	End of the loop.
17	`System.out.println(result);`	Print (X^Y) % P.
18	`}`	End of main.
19	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Input: X = 2, Y = 3, P = 5. result = 1 % 5 = 1 and x = 2 % 5 = 2. Y = 3 is 11 in binary.

Loop	y	y odd?	result after	x after (x*x % p)	y after
1	3	Yes	(1 * 2) % 5 = 2	(2 * 2) % 5 = 4	1
2	1	Yes	(2 * 4) % 5 = 3	(4 * 4) % 5 = 1	0

Result

`y = 0 ends the loop. 2^3 = 8 and 8 % 5 = 3. Output: 3`

Question 10.2

Prime Query using Sieve & Prefix Sums

Marks: 10 | Bloom's Taxonomy Level: L3 (Apply)

1 Problem

Given Q queries, each query containing two numbers L and R. Find the count of prime numbers in the range [L, R] inclusive.

Constraints / Requirements

- Precompute primes using the Sieve of Eratosthenes.
- Use a prefix sum array to answer each query in O(1) time.
- Time Complexity: O(N log log N + Q) where N = 10^6.

Input Format

First line: Integer Q (number of queries).
Next Q lines: Two space-separated integers L and R.

Output Format

Q lines, each containing the count of primes for that query.

Sample Input	Sample Output
2 1 10 10 20	4 4

2 Java Solution

```
 1  import java.util.*;
 2
 3  public class Main {
 4      public static void main(String[] args) {
 5          int N = 1000000;
 6          boolean[] composite = new boolean[N + 1];
 7          composite[0] = true;
 8          composite[1] = true;
 9          for (int i = 2; (long) i * i <= N; i++) {
10              if (!composite[i]) {
11                  for (int j = i * i; j <= N; j += i) composite[j] = true;
12              }
13          }
14          int[] prefix = new int[N + 1];
15          for (int i = 1; i <= N; i++) {
16              prefix[i] = prefix[i - 1] + (composite[i] ? 0 : 1);
17          }
18
19          Scanner sc = new Scanner(System.in);
20          int q = sc.nextInt();
21          StringBuilder sb = new StringBuilder();
22          for (int i = 0; i < q; i++) {
23              int l = sc.nextInt();
24              int r = sc.nextInt();
25              if (l < 1) l = 1;
26              sb.append(prefix[r] - prefix[l - 1]).append("\n");
27          }
28          System.out.print(sb);
29      }
30  }
```

3 Line-by-Line Explanation

Line	Code	Explanation
1	`import java.util.*;`	Import Scanner for reading input.
3	`public class Main {`	The class that holds the program.
4	`    public static void main(String[] args) {`	The program starts running here.
5	`        int N = 1000000;`	Upper limit of the sieve.
6	`        boolean[] composite = new boolean[N + 1];`	composite[i] is true when i is not prime (false at the start).
7	`        composite[0] = true;`	0 is not prime.
8	`        composite[1] = true;`	1 is not prime.
9	`        for (int i = 2; (long) i * i <= N; i++) {`	Only i up to the square root of N is needed.
10	`            if (!composite[i]) {`	If i is still marked prime...
11	`                for (int j = i * i; j <= N; j += i) composite[j] = true;`	...mark its multiples (from i*i) as not prime.
12	`            }`	End of the check.
13	`        }`	End of the sieve loop.
14	`        int[] prefix = new int[N + 1];`	prefix[i] will be the number of primes from 1 to i.
15	`        for (int i = 1; i <= N; i++) {`	Build the prefix sums.
16	`            prefix[i] = prefix[i - 1] + (composite[i] ? 0 : 1);`	Previous count plus 1 if i is prime.
17	`        }`	End of the loop.
19	`        Scanner sc = new Scanner(System.in);`	Scanner for keyboard input.
20	`        int q = sc.nextInt();`	Number of queries.
21	`        StringBuilder sb = new StringBuilder();`	Collects the answers.
22	`        for (int i = 0; i < q; i++) {`	Handle each query.
23	`            int l = sc.nextInt();`	Left end L.
24	`            int r = sc.nextInt();`	Right end R.
25	`            if (l < 1) l = 1;`	Protect against L = 0 (primes start at 2).
26	`            sb.append(prefix[r] - prefix[l - 1]).append("\n");`	Primes in [L, R] = prefix[R] - prefix[L-1].
27	`        }`	End of the query loop.
28	`        System.out.print(sb);`	Print one answer per line.
29	`    }`	End of main.
30	`}`	End of class Main.

4 Dry Run (Sample Test Case)

Sieve: starting at 2, mark multiples of each prime as not prime. The primes up to 20 are 2, 3, 5, 7, 11, 13, 17, 19. The table shows prefix[i] (count of primes from 1 to i) for the numbers needed.

prefix table (prefix[0] = 0)

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
prime?	N	Y	Y	N	Y	N	Y	N	N	N	Y	N	Y	N	N	N	Y	N	Y	N
prefix[i]	0	1	2	2	3	3	4	4	4	4	5	5	6	6	6	6	7	7	8	8

Answering the queries

Query	Formula	Calculation	Answer
1 10	prefix[10] - prefix[0]	4 - 0	4
10 20	prefix[20] - prefix[9]	8 - 4	4

Result

```
Output:
4
4
```